

Clase 103 — Ataques y seguridad de JWT

Parte: 4 — Seguridad de aplicaciones web · Fuente: Bug Bounty Bootcamp (Vickie Li) / RFC 7519 🕒

Duración estimada: 110 min · Nivel: Avanzado

Objetivo

Auditar la seguridad de los **JSON Web Tokens (JWT)**, hoy omnipresentes en APIs y SPAs. Aprenderás a decodificarlos, detectar implementaciones inseguras y explotar fallos clásicos: `alg:none`, confusión de algoritmos, claves débiles y falta de verificación de firma.

⚠️ **Ética:** solo en labs propios/autorizados. Forjar tokens de sistemas ajenos es un delito.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Decodificar** y comprender la estructura de un JWT (header, payload, signature).
2. **Explotar** el ataque `alg:none` y la aceptación de firma vacía.
3. **Realizar** confusión de algoritmos (RS256 → HS256).
4. **Crackear** claves HMAC débiles por fuerza bruta.
5. **Recomendar** una configuración segura de JWT.

Temas

#	Tema	Por qué importa
1	Estructura y claims de JWT	Base para atacar
2	Algoritmos: HS256 vs. RS256	El eje de varios ataques
3	<code>alg:none</code>	Bypass de firma
4	Confusión de algoritmos	Firmar con la clave pública
5	Claves HMAC débiles	Crackeo offline
6	Claims: exp, iss, aud, kid	Fallos de validación
7	Defensa: verificar firma y alg	Cierre del fallo

Definiciones y características

- **JWT:** token compacto con header, payload y firma en Base64URL. Característica: autocontenido y stateless.

- **alg:none** : algoritmo que indica "sin firma". Característica: si el servidor lo acepta, cualquiera forja tokens.
- **Confusión de algoritmos**: hacer que el servidor verifique un RS256 como HS256 usando la clave pública como secreto HMAC. Característica: explota validación laxa del **alg**.
- **kid (key id)**: cabecera que indica qué clave usar. Característica: inyectable (path traversal, SQLi) si no se valida.
- **Claim exp** : expiración del token. Característica: si no se valida, los tokens no caducan.
- **Secreto HMAC**: clave compartida en HS256. Característica: si es débil, se crackea offline.

Herramientas y preparación

- **jwt.io** (decodificar/inspeccionar) y **jwt_tool**.
- **Burp** con la extensión **JWT Editor**.
- **hashcat** o **John the Ripper** para crackear secretos.
- **PortSwigger labs** de JWT.

```
git clone https://github.com/ticarpi/jwt_tool && cd jwt_tool && pip install -r requirements.t
```

Laboratorio guiado

 Solo en labs propios.

1. Captura un JWT y decodifícalo con **jwt.io** o **jwt_tool** para ver header y claims.
2. Prueba el ataque **alg:none**: cambia `"alg": "none"`, elimina la firma y modifica un claim (p. ej. `role: admin`).
3. Si usa RS256, intenta **confusión de algoritmos**: firma un HS256 usando la clave pública del servidor como secreto.
4. Si es HS256, extrae el token y **crackea el secreto** con hashcat:

```
hashcat -m 16500 token.txt wordlist.txt
```

5. Con el secreto, forja un token con privilegios elevados y úsalo.
6. Manipula el **kid** para apuntar a un archivo/valor controlado.
7. Verifica el manejo de **exp**: reusa un token caducado.

Ejercicios

1. Explica cada parte de un JWT y qué contiene el payload.
2. Reproduce el ataque **alg:none** en un lab de PortSwigger.
3. Realiza confusión RS256→HS256 y explica por qué funciona.
4. Crackea un secreto HMAC débil y forja un token admin.
5. Analiza los claims **exp/iss/aud** y qué pasa si no se validan.
6. Escribe la validación correcta de JWT en el lenguaje que prefieras.

Reto verificable

Resuelve un lab de JWT de PortSwigger (alg:none, clave débil o confusión de algoritmos) y **escala a administrador** forjando un token. **Criterio de aceptación:** el lab queda resuelto, entregas el token forjado, el ataque usado y la corrección (verificar firma, fijar el algoritmo esperado, secreto fuerte).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
alg:none rechazado	Servidor valida el algoritmo; prueba otro ataque
Confusión no funciona	La librería fija el alg; documenta como fortaleza
hashcat no crackea	Secreto fuerte; solo funciona con claves débiles
Token modificado rechazado	La firma sí se verifica; revisa el vector
kid no explotable	Se valida contra allowlist; correcto

Preguntas frecuentes

 **¿JWT es inseguro?** No por sí mismo. Los fallos vienen de implementaciones que no verifican bien la firma o el algoritmo.

 **¿Puedo revocar un JWT?** No fácilmente por ser stateless. Se usan expiraciones cortas y listas de revocación o rotación de claves.

 **¿Guardo el JWT en localStorage o cookie?** Cookie HttpOnly reduce el robo vía XSS; localStorage es accesible por JS. Cada opción tiene trade-offs de CSRF/XSS.

Referencias

- Li, *Bug Bounty Bootcamp*, sección de JWT.
- RFC 7519 (JWT): <https://datatracker.ietf.org/doc/html/rfc7519>
- OWASP JWT Cheat Sheet.
- PortSwigger JWT: <https://portswigger.net/web-security/jwt>

Siguiente clase

Clase 104 - Seguridad de OAuth 2.0 y OpenID Connect